

# Nifty 100 Financial Intelligence Platform

## Analyst Guide

[Screener usage](#) · [Dashboard navigation](#) · [PDF reports](#) · [REST API](#) · [Troubleshooting](#)

# Table of Contents

- 1. Getting Started
- 2. Using the Investment Screener
- 3. Dashboard Screen-by-Screen Guide
- 4. Generating PDF Tearsheets
- 5. Calling the REST API
- 6. Troubleshooting Common Issues
- 7. Known Data Limitations (Read Before Interpreting Results)

# 1. Getting Started

This platform analyzes 92 Nifty 100 companies using data loaded into a local SQLite database (data/nifty100.db). All commands below assume a Windows 11 / PowerShell environment with the project's virtual environment (.venv) activated.

## Activate the environment:

```
.\.venv\Scripts\Activate.ps1
```

## Rebuild the database from source Excel files (if needed):

```
python -m src.etl.full_load
```

## Recompute all financial ratios (after any KPI formula change):

```
python -m src.analytics.populate_financial_ratios
```

## Run the full test suite:

```
python -m pytest tests/ --html=reports/pytest_report.html --self-contained-html
```

Note: use **python -m ...** for every module invocation, not **py -m ...** — the **py** launcher can point to a stale interpreter path independent of the active venv on some machines. If **python** itself fails to resolve, close and reopen the terminal, reactivate the venv, and try again before troubleshooting further.

## 2. Using the Investment Screener

The screener is available two ways: inside the Streamlit dashboard (Screener screen, sidebar sliders) and via the REST API (**GET /api/v1/screener**). Both use the same underlying logic and return identical results for the same filter values — this was explicitly verified (Sprint 6 Day 42 integration test).

### Six preset screeners are available:

| Preset              | Filters                                                     | Typical count |
|---------------------|-------------------------------------------------------------|---------------|
| Quality Compounder  | ROE>15%, D/E<1 (Financials exempt), FCF>0, Rev CAGR 5yr>10% | ~21           |
| Value Pick          | P/E<50, P/B<5                                               | ~14           |
| Growth Accelerator  | PAT CAGR 5yr>20%, Rev CAGR 5yr>15%, D/E<2                   | ~19           |
| Dividend Champion   | Dividend Yield>3.5%, Payout<80%, FCF>0                      | ~14           |
| Debt-Free Blue Chip | D/E<0.1, Financials excluded entirely, ROE>12%              | ~19           |
| Turnaround Watch    | Revenue CAGR 3yr>18%, FCF improving, D/E declining          | ~12           |

**Important:** preset thresholds were recalibrated from the original project spec using real data distributions (e.g. Value Pick's P/E threshold was raised from 20 to 50, since Nifty 100 large-caps trade at higher multiples than the spec assumed). See docs/PROGRESS.md Sprint 3 Day 16 for the full rationale on each recalibration.

### Custom filters:

Beyond the presets, any combination of filters can be built via the dashboard's sidebar sliders (Financial Screener screen) or the API's query parameters: min\_roe, max\_de, min\_fcf, sector, min\_rev\_cagr\_5yr, min\_pat\_cagr\_5yr, max\_pe.

### 3. Dashboard Screen-by-Screen Guide

Start the dashboard with: **streamlit run src/dashboard/app.py** — it opens at <http://localhost:8501>. All 8 screens are listed in the left sidebar.

#### Home / Overview

Summary KPI tiles (avg ROE, median P/E), sector donut chart, top-5 composite score table. Sector filter chips and year selector at the top.

#### Company Profile

Search by ticker or company name. Shows a company card, 6 KPI tiles, 10-year revenue/profit chart, ROE/ROCE trend line, and pros/cons badges.

#### Financial Screener

10 metric sliders in the sidebar, live-updating results table, 6 preset buttons, and a CSV download button.

#### Peer Comparison

Select a peer group from the dropdown to see a radar chart plus a side-by-side comparison table with the group's designated benchmark company highlighted.

#### Trend Analysis

Search a company, select up to 3 metrics to overlay on a 10-year line chart with year-over-year percentage change annotations.

#### Sector Analysis

Select a sector to see a bubble chart (revenue vs. ROE, sized by market cap) and a bar chart of sector median KPIs.

#### Capital Allocation Map

Treemap of all 92 companies grouped into 8 capital allocation patterns (Reinvestor, Shareholder Returns, Distress Signal, etc.). Click a pattern to drill into the company list.

#### Annual Reports

Search a company to see clickable links to its historical annual report PDFs on BSE India. A red badge marks years with no available report.

Note: the Sector Analysis screen shows 10 sectors, not 11 as an earlier project specification assumed — the underlying sectors table genuinely contains only 10 distinct values. This is called out directly in-app on that screen.

## 4. Generating PDF Tearsheets

Each of the 92 companies has a 2-page PDF tearsheet: KPI tiles, revenue/profit and ROE/ROCE charts on page 1; balance sheet composition, cash flow waterfall, and pros/cons on page 2.

### Generate a single company's tearsheet:

```
python -m src.reports.tearsheet --ticker TCS
```

### Generate the full batch (all 92):

```
python -m src.reports.batch_tearsheets
```

Output goes to reports/tearsheets/<TICKER>\_tearsheet.pdf. One company (JIOFIN) is deliberately skipped — it has only 2 years of trading history, below the 3-year minimum needed for a meaningful trend chart. This is logged to output/skipped\_tearsheets.csv, not a failure.

### Also available:

```
python -m src.reports.sector_report    (11-sector median KPI reports; note: spec  
says 11, real data produces 10)  
python -m src.reports.portfolio_summary (all 92 companies, 1 page each)
```

## 5. Calling the REST API

Start the API with: **python -m uvicorn src.api.main:app --port 8000** — interactive OpenAPI docs are then available at <http://localhost:8000/docs>.

### Health check

```
curl.exe http://localhost:8000/api/v1/health
```

### List companies in a sector

```
curl.exe "http://localhost:8000/api/v1/companies?sector=Information%20Technology"
```

### Full company profile

```
curl.exe http://localhost:8000/api/v1/companies/TCS
```

### P&L; history with year filter

```
curl.exe "http://localhost:8000/api/v1/companies/TCS/pl?from_year=2022-03&to_year=2024-03"
```

### Run the screener

```
curl.exe "http://localhost:8000/api/v1/screener?min_roe=15&max_debt=1&sector=Information%20Technology"
```

### Peer group comparison

```
curl.exe "http://localhost:8000/api/v1/peers/IT%20Services"
```

### Download a tearsheet PDF

```
curl.exe -o tcs_tearsheet.pdf  
http://localhost:8000/api/v1/companies/TCS/tearsheet
```

### Portfolio-wide statistics

```
curl.exe http://localhost:8000/api/v1/portfolio/stats
```

**Data quality flags on the API:** a small number of companies (HAL, BEL, INDIGO, ICICIPRULI, HDFCLIFE) have ROE/ROCE/ROA values so extreme they are almost certainly data artifacts, not real performance (e.g. one company's raw ROE computes to over 3,800%). Rather than silently hiding these, the API returns **null** for the field plus an explicit sibling field, e.g. **return\_on\_equity\_pct\_quality\_flag: "excluded\_sanity\_bound\_exceeded"**, so API consumers know data exists but was excluded, rather than mistaking it for a missing value.

## 6. Troubleshooting Common Issues

### "py" command not found or points to a broken interpreter

Use **python -m ...** instead of **py -m ...** for every command in this guide. The **py** launcher's registered interpreter path can go stale independently of the project's own `.venv`, especially after a Python reinstall.

### ModuleNotFoundError when running the full test suite together

A small number of older test files use **from analytics.X import ...** while newer files use **from src.analytics.X import ...**. These files pass individually (**pytest tests/kpi/**) but fail to collect when the entire `tests/` tree runs together. Workaround: run **pytest tests/ --continue-on-collection-errors**, which correctly reports all other tests' pass/fail status and lists the affected files separately as collection errors, not test failures.

### API returns HTTP 500 under concurrent load

Fixed as of Sprint 6 Day 43: `src/api/db.py`'s database connection now uses **check\_same\_thread=False**. If you see "SQLite objects created in a thread can only be used in that same thread", confirm this fix is present in your checkout.

### A company shows blank or "N/A" for ROE, ROCE, or D/E

Check whether the company is SBIN (zero balance sheet rows — a genuine source data gap, not a bug) or one of HAL/BEL/INDIGO/ICICIPRULI/HDFCLIFE (values excluded by the sanity-bound check because they are implausibly extreme). Both cases are intentional, documented data-quality handling, not application errors.

### Dashboard chart looks empty or shows a warning for a metric

Some companies have long stretches of missing data for a specific metric across all reported years (SBIN's ROE/ROCE/D/E, for example). The dashboard shows an explicit "no plottable data" message in this case rather than a blank or broken chart.

### Report file smaller than expected

A company missing one or more chart inputs (e.g. SBIN's balance sheet chart, or the sanity-bound-masked companies' ROE/ROCE chart) will produce a correctly smaller PDF with 3 of 4 charts instead of 4 — this is expected given the known data gap, not a rendering failure.



## 7. Known Data Limitations (Read Before Interpreting Results)

This platform is built directly against the real source data, including its imperfections. The following are established, investigated findings — not unexplained bugs — documented in full in docs/PROGRESS.md.

- SBIN has zero rows in the balance sheet source file. All balance-sheet-derived metrics (ROE, ROCE, D/E, Asset Turnover, Book Value/Share) are unavailable for SBIN specifically.
- HAL's balance sheet equity figures are roughly 147x too small relative to independently sourced figures, for reasons not fully diagnosed. This produces implausible ROE/ROCE; these fields are excluded (not fabricated) wherever HAL appears.
- BEL, INDIGO, ICICIPRULI, and HDFCLIFE show the same category of implausible ROE/ROCE for similar near-zero-equity reasons; all are excluded the same way.
- The sectors table contains 10 distinct broad sectors, not the 11 originally specified. Root cause not diagnosed; the platform reports the real 10.
- Operating margin should always be read from the computed value (sales minus expenses divided by sales), never from the source file's own opm\_percentage field, which diverges from the computed value for a number of companies, including several where it appears to hold a raw currency figure rather than a percentage.
- Financial-sector companies (banks, NBFCs, insurers) are exempted from several cross-sector rules (high D/E flags, ROCE benchmark bands, a cash-flow distress rule) because high leverage and unusual cash flow shapes are structurally normal for lenders, not warning signs.

For the complete, day-by-day record of every data-quality finding and design decision behind this platform, see docs/PROGRESS.md.